

ProjectManagerApp — Hướng Dẫn 04: Utils, Session, Exception

Package: `com.projectmanager.exceptions` · `com.projectmanager.utils` · `com.projectmanager.session`

AppException.java

```
package com.projectmanager.exceptions;

// Custom unchecked exception – dùng khi lỗi nghiệp vụ nội bộ (hash, parse)
// Khác với IllegalArgumentException (dùng cho lỗi do user nhập sai input)
public class AppException extends RuntimeException {

    public AppException(String message) {
        super(message);
    }

    public AppException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

AppException vs IllegalArgumentException:

- `IllegalArgumentException` → caller truyền sai tham số (validate input từ UI)
- `AppException` → lỗi hệ thống nội bộ (SHA-256 không khả dụng, parse lỗi không do user)

PasswordHasher.java

```
package com.projectmanager.utils;

import com.projectmanager.exceptions.AppException;
import java.security.MessageDigest;
import java.util.Base64;

// SHA-256 hash – dùng java.security built-in, không cần thư viện ngoài
// Trong production: dùng BCrypt (bcrypt-java lib) thay SHA-256 vì bcrypt có salt
// SHA-256 đủ cho mục đích học – dễ hiểu, không cần dependency ngoài
public class PasswordHasher {

    private PasswordHasher() {} // utility class – không khởi tạo

    public static String hash(String password) {
        try {
```

```

        MessageDigest md      = MessageDigest.getInstance("SHA-256");
        byte[]          digest = md.digest(password.getBytes("UTF-8"));
        return Base64.getEncoder().encodeToString(digest);
    } catch (Exception e) {
        throw new ApplicationException("Hash error: " + e.getMessage(), e);
    }
}

public static boolean verify(String plainPassword, String storedHash) {
    return hash(plainPassword).equals(storedHash);
}
}

```

Hash tính trước để seed DB:

Password	SHA-256 (Base64)
admin123	jGl25bVBBBW96Qi9Te4V37Fnqchz/Eu4qB9vKrRIqRg=
user123	pmWkWSBCL51Bfkhn79xPuKBKHz//H6B+mY6G9/eieuM=

```

// Chạy 1 lần để lấy hash seed DB:
public static void main(String[] args) {
    System.out.println>PasswordHasher.hash("admin123"));
    System.out.println>PasswordHasher.hash("user123"));
}

```

Validator.java

```

package com.projectmanager.utills;

// Static utility – không khởi tạo
// Ném IllegalArgumentException với message rõ ràng → Controller bắt và hiển thị
trên Label
public class Validator {

    private Validator() {}

    public static void requireNotBlank(String value, String field) {
        if (value == null || value.isBlank())
            throw new IllegalArgumentException(field + " không được để trống.");
    }

    public static void requireMinLength(String value, String field, int min) {
        if (value != null && value.length() < min)
            throw new IllegalArgumentException(field + " phải có ít nhất " + min +
" ký tự.");
    }
}

```

```

    public static void requirePositive(double value, String field) {
        if (value <= 0)
            throw new IllegalArgumentException(field + " phải lớn hơn 0.");
    }

    public static void requireRange(int value, String field, int min, int max) {
        if (value < min || value > max)
            throw new IllegalArgumentException(field + " phải từ " + min + " đến "
+ max + ".");
    }

    public static void requireOneOf(String value, String field, String... options)
{
        for (String opt : options) {
            if (opt.equalsIgnoreCase(value)) return;
        }
        throw new IllegalArgumentException(field + " phải là một trong: " +
String.join(", ", options));
    }

    // Parse và validate cùng lúc – ném IllegalArgumentException thay vì
    NumberFormatException
    public static int parsePositiveInt(String raw, String field) {
        try {
            int v = Integer.parseInt(raw.trim());
            if (v <= 0) throw new IllegalArgumentException(field + " phải lớn hơn
0.");

            return v;
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(field + " phải là số nguyên hợp
le.");
        }
    }
}

```

Cách dùng trong AddTaskController:

```

try {
    Validator.requireNotBlank(id, "Task ID");
    Validator.requireNotBlank(title, "Tiêu đề");
    Validator.requireOneOf(priority, "Priority", "LOW", "MEDIUM", "HIGH");
    if (isBug) Validator.requireOneOf(severity, "Severity",
"LOW", "MEDIUM", "HIGH", "CRITICAL");
    else hours = Validator.parsePositiveInt(txtHours.getText(), "Số giờ");
    // ...
} catch (IllegalArgumentException e) {
    showMsg(e.getMessage(), false); // hiển thị trên lblMessage màu đỏ
}

```

UserSession.java — Singleton Pattern (lần 3)

```
package com.projectmanager.session;

import com.projectmanager.models.entity.User;

//
// | SINGLETON PATTERN – static field (không cần instance) |
// | Khác DatabaseConfig: không có object nào được tạo   |
// | Chỉ dùng static fields và static methods             |
// | Lợi ích: mọi Controller đọc UserSession.get() mà không |
// | cần truyền User qua constructor hay parameter        |
//
public class UserSession {

    private static User currentUser;

    private UserSession() {}

    public static void    set(User user) { currentUser = user; }
    public static User    get()          { return currentUser; }
    public static boolean isLoggedIn()   { return currentUser != null; }

    // Kiểm tra role – TaskListCtrl dùng để ẩn/hiện nút Delete
    // DashboardCtrl dùng để ẩn/hiện nút "Manage Users"
    public static boolean isAdmin() {
        return currentUser != null && "admin".equalsIgnoreCase(currentUser.role);
    }

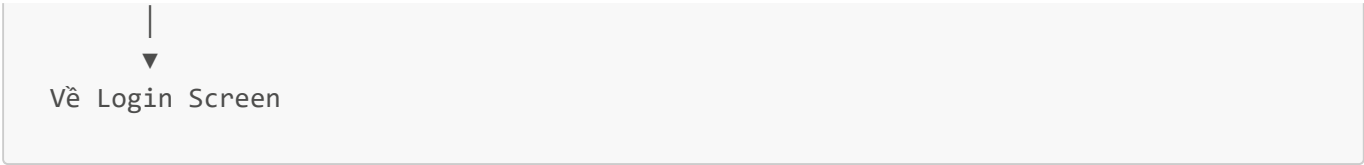
    // Gọi khi logout – xóa session và reset service cache
    public static void clear() { currentUser = null; }
}
```

Vòng Đời Session:

```

Khởi động app
  |
  ▼
Login thành công
  UserSession.set(user)      ← lưu user vào static field
  |
  ▼
Dashboard / AddTask / TaskList / UserList
  UserSession.get()          ← đọc từ bất kỳ controller nào
  UserSession.isAdmin()      ← kiểm tra quyền
  |
  ▼
Logout
  UserSession.clear()        ← xóa static field
  ProjectService.reset()     ← xóa cache service (tránh data rò rỉ)

```



Ghi Chú Thiết Kế Utils

Class	Kiểu	Mục đích
AppException	Custom RuntimeException	Lỗi hệ thống nội bộ (hash, system error) — không phải lỗi do user nhập
PasswordHasher	Static utility	SHA-256 + Base64 encode/verify password
Validator	Static utility	Validate UI input → ném <code>IllegalArgumentException</code> với message UI-friendly
UserSession	Static Singleton	Giữ trạng thái login xuyên suốt app — không truyền qua constructor

Luồng Xử Lý Lỗi Đầy Đủ

```
LoginController.handleLogin()
├─ try { authService.login(req) }
│   └─ Validator.requireNonBlank() → throws IllegalArgumentException
│       ↓ catch → showMsg(e.getMessage(), false) ← màu đỏ
│   └─ PasswordHasher.hash() → throws AppException (SHA-256 lỗi)
│       ↓ catch(Exception e) → showMsg("System error: " + ..., false)
└─ authService.login() trả về null
    → showMsg("Sai username hoặc mật khẩu", false)
```